

# Atividade — Comandos Iniciais Linux

Navegação, manipulação de arquivos, edição, redirecionamento, pipes e gerenciamento de pacotes no Linux Mint

|                           |                                                        |
|---------------------------|--------------------------------------------------------|
|                           |                                                        |
| <b>Disciplina</b>         | Análise e Desenvolvimento de Sistemas — Cloud & DevOps |
| <b>Modalidade</b>         | Individual                                             |
| <b>Ambiente</b>           | VirtualBox + Linux Mint                                |
| <b>Formato de entrega</b> | Arquivo <code>.txt</code> + PDF de respostas           |

---

Em vez de listas soltas de comandos, vocês vão simular a organização inicial de um projeto para uma empresa fictícia — a **CloudMint Tech** — e produzir, ao final, evidências objetivas de que o trabalho foi realmente executado no terminal de vocês.

**Por que a entrega precisa ter "prova"?** Em um time real de DevOps, ninguém confia apenas na palavra de que um comando "funcionou" — existem logs, históricos e registros de execução para isso. Esta atividade usa a mesma lógica: o comando `script` vai gravar, em um arquivo de texto, tudo o que acontece no terminal durante a atividade — inclusive os erros, caso aconteçam.

---

## Passo 0 — Iniciando o registro da sessão

Antes de começar qualquer tarefa, abram o terminal e rodem o comando abaixo. Ele cria um arquivo chamado `relatorio-aula-inicio.txt` que vai gravar, do início ao fim, tudo o que for digitado e exibido no terminal — esse arquivo é a base da correção.

```
cd ~  
script relatorio-aula-inicio.txt
```

**Atenção:** a partir deste momento, tudo o que vocês digitarem ficará registrado. Não fechem o terminal nem digitem `exit` até concluir a última tarefa — isso encerraria a gravação antes da hora.

---

## Parte 1 — Organizando o projeto (FHS na prática)

### Tarefa 1 — Estrutura do projeto

Dentro do seu diretório pessoal, crie a estrutura abaixo **com um único comando** usando

```
mkdir -p:  
cloudmint/src  
cloudmint/config  
cloudmint/logs  
cloudmint/docs
```

**Prova exigida:** rodar `tree cloudmint` logo após criar a estrutura, para que a árvore de pastas fique registrada no relatório.

### Tarefa 2 — Justificando a organização

No arquivo de respostas (Parte 5 deste documento), expliquem: por que faz sentido separar `src`, `config`, `logs` e `docs` em pastas diferentes, em vez de colocar tudo junto? Relacionem com a lógica do FHS vista na Parte 1 do texto didático (por exemplo, com o papel de `/etc` e `/var/log` no sistema real).

---

## Parte 2 — Manipulação, visualização e edição

### Tarefa 3 — Criando e populando arquivos

Dentro de `cloudmint/src`, criem um arquivo chamado `app.py`. Usando o `nano`, escrevam dentro dele exatamente estas três linhas:

```
print("CloudMint Tech - servico iniciado")
print("Ambiente: producao")
print("Autor: <coloquem seu nome aqui>")
```

Depois, copiem esse arquivo para dentro de `docs` com o nome `app-backup.py`, e renomeiem o arquivo original em `src` para `main.py` (usando `mv`).

**Prova exigida:** rodar `cat` nos dois arquivos finais (`src/main.py` e `docs/app-backup.py`) para que o conteúdo apareça no relatório.

### Tarefa 4 — Gerando e explorando um log simulado

Gerem um arquivo de log fictício com 300 linhas numeradas dentro de `cloudmint/logs`:

```
seq 1 300 > cloudmint/logs/sistema.log
```

Agora respondam usando os comandos corretos (não vale abrir o arquivo inteiro com `cat`):

Mostrem apenas as **10 primeiras** linhas do log

Mostrem apenas as **15 últimas** linhas do log

Usando `less`, abram o arquivo, naveguem até o meio e saiam com `q`

**Prova exigida:** os comandos de `head` e `tail` executados (com a saída visível) ficam automaticamente no relatório.

---

## Parte 3 — Redirecionamento, pipes e automação

### Tarefa 5 — Filtro com resultado verificável

A partir do arquivo `cloudmint/logs/sistema.log` criado na tarefa anterior, produzam um novo arquivo chamado `cloudmint/logs/filtrado.log` contendo **apenas os números que terminam em 7** (ex.: 7, 17, 27, 37...), usando `grep` combinado com redirecionamento:

```
cat cloudmint/logs/sistema.log | grep "7$" >
cloudmint/logs/filtrado.log
```

O `$` depois do 7 indica "fim da linha" — ou seja, o número precisa *terminar* em 7, e não apenas conter o dígito em qualquer posição. Depois, contem quantas linhas o arquivo resultante possui:

```
wc -l cloudmint/logs/filtrado.log
```

**Prova exigida:** o resultado da contagem deve aparecer no relatório. Como o intervalo é de 1 a 300, existe um número exato e verificável de valores terminados em 7 — quem entregar um valor diferente do esperado terá o comando revisado na correção.

### Tarefa 6 — Registro cumulativo

Usando `>>`, adicionem ao arquivo `cloudmint/docs/changelog.txt` três linhas, uma por vez, em três comandos separados:

```
echo "v1.0 - estrutura inicial do projeto criada" >>
cloudmint/docs/changelog.txt
echo "v1.1 - modulo principal implementado" >>
cloudmint/docs/changelog.txt
echo "v1.2 - log de sistema configurado" >>
cloudmint/docs/changelog.txt
```

**Prova exigida:** rodar `cat cloudmint/docs/changelog.txt` ao final e confirmar que as três linhas aparecem na ordem correta.

## Parte 4 — Atualização, instalação e checagem final

### Tarefa 7 — Preparando o ambiente

```
sudo apt update  
sudo apt install tree  
sudo apt install htop
```

*(Se algum dos dois já estiver instalado, o próprio terminal vai indicar isso — registrem a mensagem mesmo assim.)*

### Tarefa 8 — Conferência final do projeto

Rodem o comando abaixo para gerar a "foto final" do projeto inteiro:

```
tree cloudmint
```

**Prova exigida:** a árvore final deve mostrar todas as pastas e arquivos criados ao longo da atividade (Tarefas 1, 3, 4, 5 e 6).

### Encerrando o registro

Depois de concluir todas as tarefas, encerrem a gravação com:

```
exit
```

Isso finaliza o arquivo `relatorio-aula-inicio.txt` na pasta pessoal de vocês — é ele que será anexado à entrega.

---

## Parte 5 — Questões de reflexão (respostas escritas)

Em um documento separado (Word ou PDF), respondam com suas próprias palavras:

| Nº | Pergunta                                                                                                                                                                                                                                                      |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Por que separar <code>src</code> , <code>config</code> , <code>logs</code> e <code>docs</code> em pastas diferentes reflete a mesma lógica do FHS do Linux real? Cite um diretório do sistema (ex: <code>/etc</code> , <code>/var/log</code> ) como paralelo. |
| 2  | Na Tarefa 5, para que serve o <code>\$</code> no comando <code>grep "7\$"</code> ? O que aconteceria se tivessem esquecido do <code>&gt;</code> antes do nome do arquivo de saída?                                                                            |
| 3  | Qual a diferença prática entre usar <code>&gt;</code> e <code>&gt;&gt;</code> ? O que teria acontecido no <code>changelog.txt</code> da Tarefa 6 se, por engano, vocês tivessem usado <code>&gt;</code> nas três linhas?                                      |
| 4  | Por que instalar programas com <code>sudo apt install</code> em vez de baixar um instalador manualmente é a prática padrão em servidores de produção?                                                                                                         |

---

### O que deve ser entregue

O arquivo `relatorio-aula-inicio.txt` gerado pelo comando `script` (sem edições manuais)

Um documento com as respostas das 4 questões de reflexão (Word ou PDF)

Nome completo e turma identificados em ambos os arquivos

**Sobre a integridade do arquivo `relatorio-aula-inicio.txt`:** ele é a evidência de que os comandos foram realmente executados — não copiem e cole comandos de colegas para dentro dele. Divergências entre o relatório e as respostas escritas (por exemplo, uma contagem de linhas incompatível com o log gerado) serão tratadas como indício de que a atividade não foi executada na própria máquina.